Data Publishing with `washr`

Table of contents

Welcome

Publishing data can be challenging, especially when adhering to standards of reproducibility. Although technically available, an Excel workbook with multiple tabs, tucked away in an online archive, is far from practical. In other words, it lacks the key principles of being findable, accessible, interoperable, and reproducible — commonly known as FAIR.

This guide aims to provide a detailed walkthrough of how data can be published according to the FAIR principles. It builds on `washr`, an R package developed for swift data publication. The package emerged from the need to streamline certain steps when publishing the datasets collected during GHE's openwashdata academy.

The guide follows a chronological structure, starting from an empty repository and resulting in the publication of data as a website. Chapter 1 introduces version control, guiding readers through setting up both local and remote repositories to track all development steps. Chapter 2 focuses on transforming the dataset into a tidy format. Chapter 3 covers the documentation of the tidy dataset. Chapter 4 explains how users can create a website to showcase their dataset. Finally, Chapter 5 helps you create a Digital Object Identifier (DOI) for your published data.

This book is a work in progress. As we publish more data, we learn more and these insights flow back into this book. If something is unclear or incomplete, please open an issue here and we'll try to address it.

1 Creating a repository

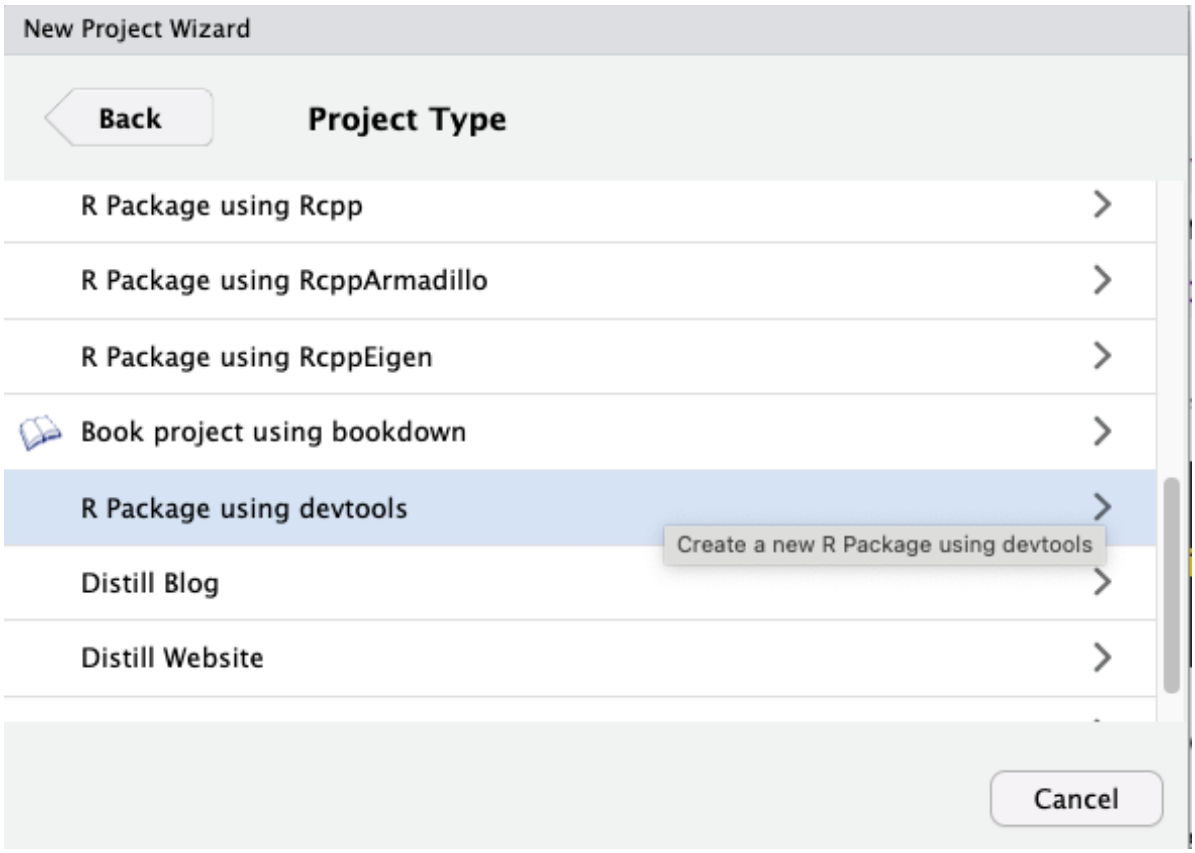
> **Note**
>
> The following steps assume that you are a member of GHE on GitHub. If you are not, reach out here so we can create a repository for you.

As your very first step, you have to create a new repository within the Global Health Engineering (hereafter referred to as GHE) organization on GitHub. You can do this here.

Choose a name for your repository, make it public if you're data is not confidential/sensitive and click on "Create repository". For now, you don't need a README, .gitignore file or a license. You'll add these at a later stage.

Once the repository is initialized, go back to RStudio and check if the `devtools` and `usethis` packages are installed. If not, please install/update them now as some of their functions are needed in the next steps.

Next, you have to create an R Project. Go to *File > New Project > New Directory > R Package using devtools*.



Copy and paste the name you chose for your GitHub repository as the directory name and select a location on your computer for your new project.

> **Note**
>
> For the next steps, we assume that you've already configured Git on your computer. If not, please make sure to do that before proceeding.

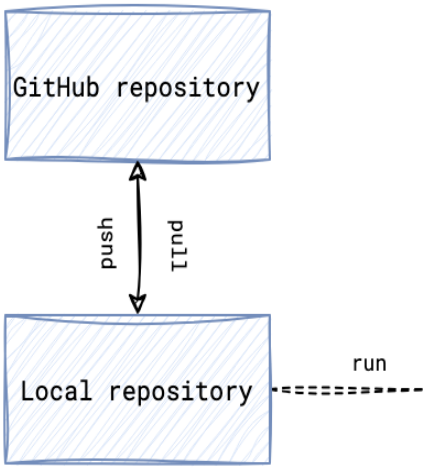
With your project set up locally and a new repository online, it's time to connect the two. To do this, switch to the Terminal (the tab next to Console in RStudio) and run the following commands one after the other.

> **Warning**
>
> Replace `REPO` with the name of your newly created repository.

```
git remote add origin "https://github.com/Global-Health-Engineering/REPO.git"
```

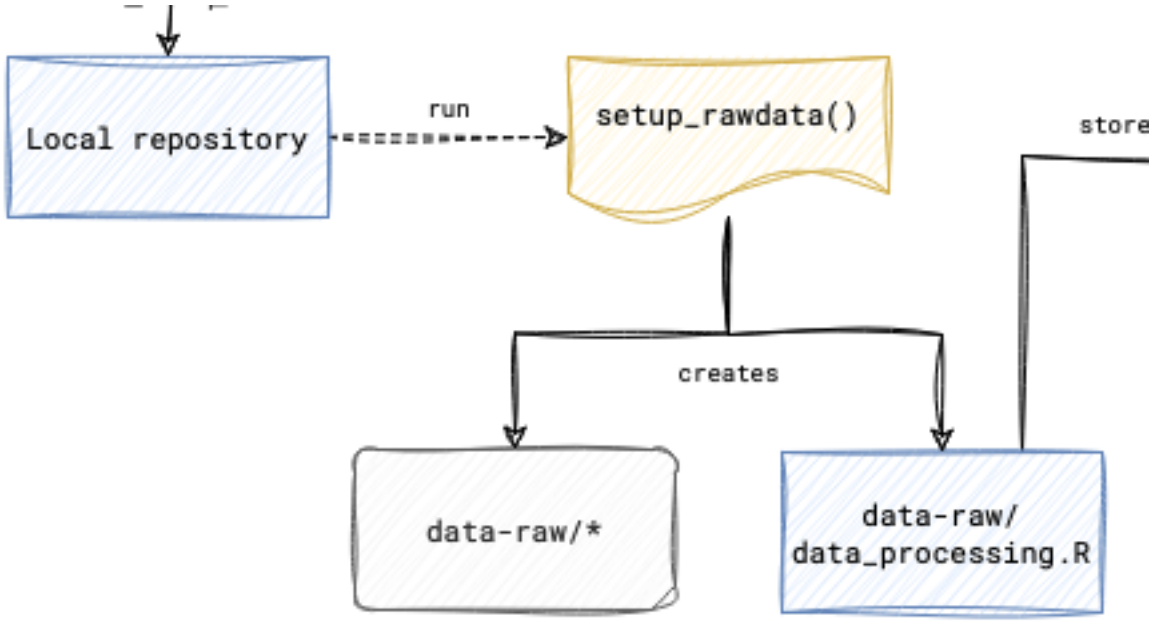
```
git branch -M main

git push -u origin main
```

If everything went well, you're local repository (on your computer) should now be connected to the remote repository (on GitHub) which allows you to push and pull changes and to invite collaborators.



2 Preparing your dataset

Everything starts with a dataset that you want to publish. You might even have several datasets, some stored locally on your computer, others on an external hard drive or in the cloud.



As a first step, you need to create a data directory where all datasets are stored in one place. To do this, open your R console within your project and type and execute the following command:

```
library(washr)
setup_rawdata()
```

This command will create a new directory called `data-raw` in your project folder where you will store all your raw datasets. Copy or move all your files into the newly created `data-raw` directory.

Next, you'll want to have this directory under version control so that you can track your changes.

In RStudio, locate the "Git" tab in the top-right panel. As a good practice, begin by clicking "Pull" to ensure you have the latest changes. In the Git tab, you will see a list of changed files—tick the checkboxes next to all the files you've added or modified. Next, click the "Commit" button. A new window will appear where you can enter a descriptive commit message, explaining what changes you've made (for example, "Add raw data files to data-raw directory"). Finally, click the "Push" button to upload your changes to GitHub.

2.1 Clean and Process Your Raw Data

Open the file `data-raw/data_processing.R` in your RStudio editor, which contains some pre-populated code. You'll need to modify this script to fit your specific data cleaning needs. Add your own R code to clean and process your raw data files, ensuring that the script reads the files from the data-raw directory and outputs a tidy, well-organized version of the data.

After you've finished editing data_processing.R, run the entire script to execute your data cleaning process. You can do this by clicking "Source" at the top of the editor window, or by selecting all the code and clicking "Run." This will process your raw data and produce a tidy version of the dataset, ready for analysis.

Return to the "Git" tab in RStudio, where you should see your modified data_processing.R file, along with any new output files that may have been generated. Commit these changes by entering a meaningful commit message, such as "Clean and process raw data." Once you've committed the changes, push them to GitHub to ensure your updates are saved and accessible.

2.2 Creating a Data Dictionary

2.2.1 Set Up the Dictionary Template

In your R console, execute the command `setup_dictionary()`. This will create a new file called dictionary.csv in your data-raw directory.

2.2.2 Fill in the Data Dictionary

Next, open `data-raw/dictionary.csv` in a spreadsheet program like Excel or in RStudio's data viewer. You will see columns corresponding to each dataset and variable in your tidy data. Focus on the "description" column: for each row representing a variable, provide a clear and concise description of what that variable represents. Include relevant information such as units

of measurement, possible values, or any other pertinent details. Once you've completed this, save your changes to `dictionary.csv`.

2.2.3 Update GitHub with Your Data Dictionary

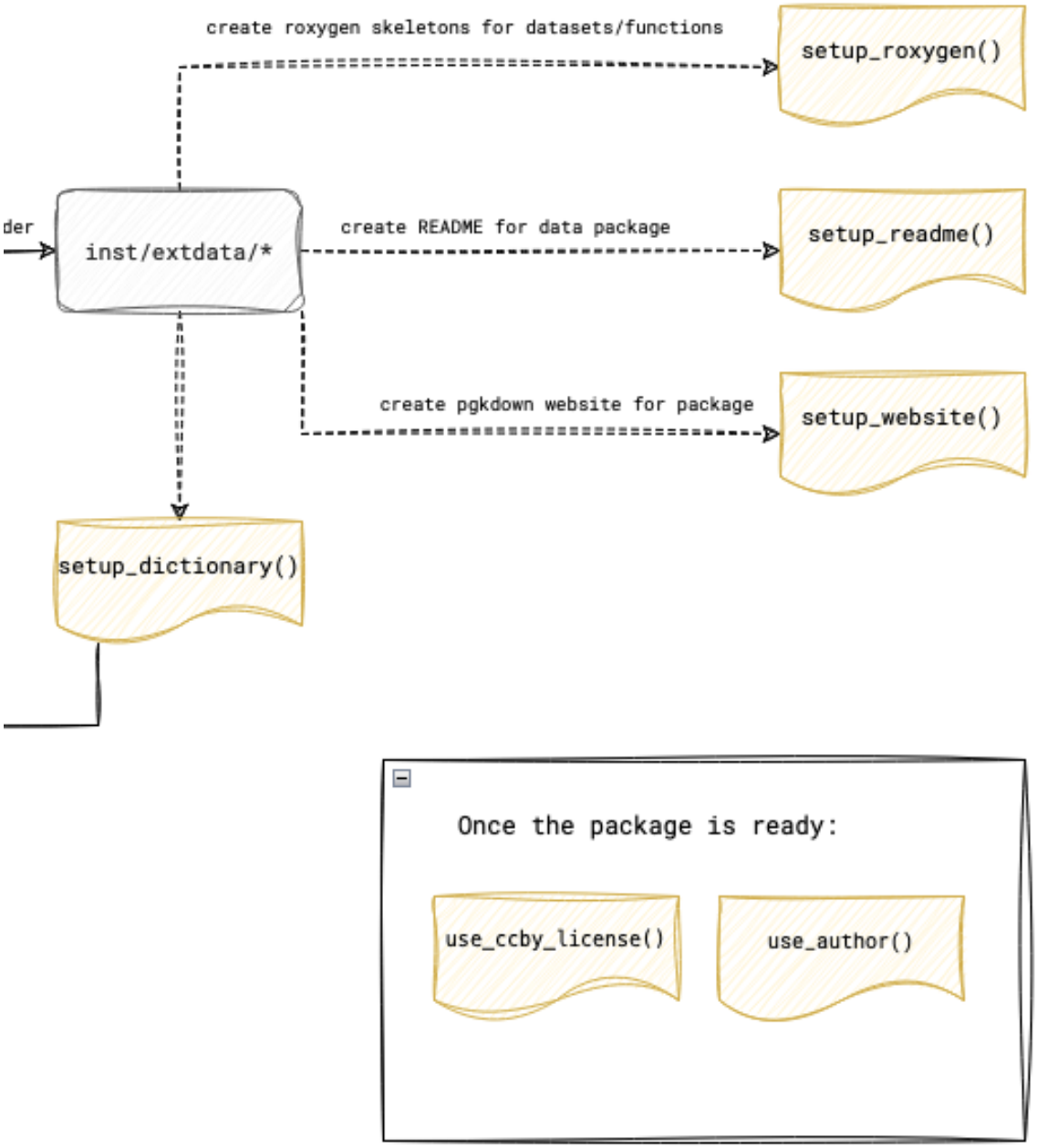
Return to the "Git" tab in RStudio, where you should see the modified `dictionary.csv` file. Commit these changes with a message like "Add data dictionary," and then push your changes to GitHub.

Finally, review your GitHub repository online to ensure that all your changes have been uploaded correctly. You should now have a `data-raw` directory containing your original data files, a `data_processing.R` script in which you clean your data, and a `dictionary.csv` file describing your tidy dataset.

Congratulations! You've successfully prepared your dataset and created a data dictionary, all while maintaining version control with Git and GitHub.

3 Documenting Your Dataset

In the next steps, the goal is to document not only the dataset itself but also the functions and the package as a whole. The individual steps are visualized in the figure below.


create roxygen skeletons for datasets/functions
setup_roxygen()
der
inst/extdata/*
create README for data package
setup_readme()
create pgkdown website for package
setup_website()
setup_dictionary()
Once the package is ready:
use_ccby_license()
use_author()

3.1 Setting Up and Writing Documentation with Roxygen

To begin documenting your dataset, start by initializing Roxygen documentation. Open your R console within the project and run the following command:

```
setup_roxygen()
```

This will create the necessary documentation files in the `R/` folder of your project, setting up the foundation for detailed documentation.

3.1.1 Writing Documentation

Once the documentation files are generated, navigate to the `R/` folder in your project. Each file in this folder, with a `.R` extension, represents a dataset or function. Open these files and provide a human-readable title and description using Roxygen comments (lines that start with `#'`). For instance, you could use the following format to document your dataset:

```
#' Title of Your Dataset
#'
#' A brief description of what the dataset contains and its purpose.
#'
#' @format A data frame with X rows and Y columns:
#' \describe{
#'   \item{column1}{Description of column1}
#'   \item{column2}{Description of column2}
#'   ...
#' }
#' @source Where the data comes from (if applicable)
"dataset_name"
```

This method allows you to provide clear, concise details about each dataset, ensuring that users understand its structure and purpose.

3.1.2 Updating Your GitHub Repository

Once you've completed the documentation, it's time to update your GitHub repository. In RStudio, go to the "Git" tab. Stage all the changed files by checking the boxes next to them, then click "Commit". Write a meaningful commit message, such as "Add Roxygen documentation for dataset", and click "Push" to update your repository with the latest changes.

3.1.3 Finalizing and Installing the Documentation

Now that your documentation is written, run the following commands in your R console:

```
devtools::document()
devtools::check()
devtools::install()
```

These commands will generate the documentation from your Roxygen comments, check for any issues in the package, and install it locally. If you receive a warning about the license, don't worry, as it will be addressed in the next section.

3.2 Updating the DESCRIPTION File

To complete your package documentation, you'll need to update the `DESCRIPTION` file with author information and other key details.

3.2.1 Adding Yourself as an Author

In your R console, add yourself as the author and maintainer by running:

```
use_author(
  given = "Your First Name",
  family = "Your Last Name",
  role = c("aut", "cre"),
  email = "your.email@example.com",
  comment = c(ORCID = "XXXX-XXXX-XXXX-XXXX")
)
```

Here, `aut` indicates that you are an author, and `cre` designates you as the creator or maintainer of the package.

3.2.2 Documenting Other Contributors

To ensure proper credit is given, create a new issue in your GitHub repository titled "Author Information for DESCRIPTION File". List all contributors, including their full name, email address, role (e.g., `aut` for authors or `ctb` for contributors), and ORCID (if applicable).

For each additional contributor, run a similar command:

```
use_author(given = "Coauthor First Name", family = "Coauthor Last Name", role = "aut")
```

3.2.3 Updating the DESCRIPTION File

After adding all contributors, open the `DESCRIPTION` file in your project and ensure the title and description reflect the purpose of your package accurately. Then, in your R console, run:

```
update_description()
```

This will update fields such as version, authors, and dependencies. Review the `DESCRIPTION` file to make sure all information is accurate.

3.2.4 Documentation Check

To complete the process, run the following commands again:

```
devtools::document()
devtools::check()
devtools::install()
```

3.3 FAIR Documentation

One function derives the machine readable metadata of your package from the files you have already written: `DESCRIPTION`, `data-raw/dictionary.csv`, `CITATION.cff` and the exports in `inst/extdata`. Nothing is edited by hand; when a value is wrong, change its source and run the function again.

3.3.1 Keywords and coverage

Three facts have no other home, so they live in `DESCRIPTION`. Open the file and add three lines, with your own values:

```
X-schema.org-keywords: sanitation, faecal sludge, Kampala
X-schema.org-spatialCoverage: Kampala, Uganda
X-schema.org-temporalCoverage: 2022-03-01/2022-09-30
```

The keywords are a comma separated list. The spatial coverage is a place name. The temporal coverage is the first and the last date of the data, separated by a slash.

3.3.2 Generating the metadata

In your R console run:

```
update_metadata()
```

The function writes a schema.org description of your dataset as JSON-LD into `pkgdown/templates/in-header.html`. The website build in the next chapter embeds it in the head of every page, where dataset search engines such as Google Dataset Search read it. The function ends with a list of the fields it could not fill and where to fill them.

Run it again after every change to `DESCRIPTION`, the dictionary or the exports, and once more after the DOI exists (see Chapter 5). A second run with nothing changed writes nothing.

3.3.3 Final Documentation Check

To complete the process, run the following commands again:

```
devtools::document()
devtools::check()
devtools::install()
```

These steps will ensure your package is fully documented, checked for errors, and ready for use.

4 Publishing Your Dataset

4.1 Creating and Editing the README

To start documenting your dataset for publication, you first need to set up a README file.

4.1.1 Initialize the README

In your R console, within the project directory, run the following command to create a README file:

```
setup_readme()
```

If you want the README to include an Example section with a scaffold for a first plot of your data, use this command instead:

```
setup_readme(has_example = TRUE)
```

The template documents the first data object in `data/`. When your package holds several, copy the data section once per further object.

4.1.2 Editing the README

Locate the `README.Rmd` file in your project directory and open it. Edit each section of the README to provide relevant information about your package. Typical sections include:

- **Package Name and Brief Description**
- **Installation Instructions**
- **Basic Usage**
- **Features**
- **Example**

4.1.3 Creating a Data Visualization

In the "Example" section of your README, it's helpful to include at least one plot that showcases your data. The section written by `setup_readme(has_example = TRUE)` holds a commented scaffold for it. For instance, using `ggplot2`, you could create a plot like this (adapt it to fit your specific data):

```
library(ggplot2)
library(yourpackagename)

ggplot(your_data, aes(x = variable1, y = variable2)) +
  geom_point() +
  theme_minimal() +
  labs(title = "Example Plot from YourPackageName")
```

4.1.4 Building the README

Once you've completed editing the README, convert it to a Markdown file by running:

```
devtools::build_readme()
```

This will generate a `README.md` file, which GitHub displays on your repository's main page.

4.1.5 Updating GitHub

After building the README, go to the "Git" tab in RStudio. Stage the newly created or modified README files, commit them with a message such as "Add and update README," and push the changes to GitHub.

4.2 Setting Up the Package Website

To enhance your package's accessibility, you can create a website for it.

4.2.1 Initializing the Website

Run the following command in your R console:

```
setup_website()
```

The first run writes `_pkgdown.yml` from the openwashdata template and builds the site into `docs/`. Every later run keeps your `_pkgdown.yml` as it is and rebuilds the site, so there is nothing to answer.

4.2.2 Applying the openwashdata brand

To give the site the openwashdata fonts and colors, run:

```
use_brand()
```

This copies the brand file and the logos from the central brand repository into your package and points `_pkgdown.yml` at them. Run it again whenever the brand changes.

4.2.3 Document, Check, and Install the Package

Ensure your package is up-to-date by running the following commands:

```
devtools::document()
devtools::check()
devtools::install()
```

These commands generate the latest documentation, check the package for issues, and install it locally.

4.2.4 Preparing for GitHub Pages

To host the package website on GitHub Pages, Git has to track the built site. `setup_website()` removes the `docs` line from `.gitignore` for you, so the `docs/` folder is tracked. Check that the line is gone before you commit.

4.2.5 Updating GitHub with Website Files

Go to the "Git" tab in RStudio. You should now see new files in the `docs/` folder and the updated `.gitignore` file. Stage these changes, commit them with a message like "Add pkgdown website files," and push the updates to GitHub.

4.2.6 Setting Up GitHub Pages

To activate the website, open your GitHub repository in a web browser. Go to "Settings" > "Pages." Under "Source," select the branch where your `docs/` folder resides (usually "main" or "master") and set the folder to `/docs`. Finally, click "Save."

Your package website will now be live at `https://yourusername.github.io/yourrepositoryname/`.

Remember, if you make significant changes to your package or documentation, you may need to rebuild the website by running `pkgdown::build_site()` and pushing the updated `docs/` folder to GitHub.

5 Creating a DOI and connecting it to GitHub

This chapter guides you through the process of creating a Digital Object Identifier (DOI) and connecting it to your GitHub repository.

5.1 Setting Up Your Repository on Zenodo

Begin by accessing your Zenodo account through your GitHub credentials. Once logged in, you'll need to establish the connection between Zenodo and your GitHub repository. To do so, navigate to the settings by clicking the dropdown menu next to your email address in the top right corner and select "GitHub" from the options.

You'll see a list of your GitHub repositories. Locate the repository you want to publish and activate it by flipping the switch to "ON". Click on the repository link to access its settings. This connection allows Zenodo to track your repository's releases and create corresponding archives.

5.1.1 Creating Your First Release

With the connection established, return to GitHub and create your initial release. Label it as version 0.0.1 and mark it as the "initial package release". This triggers Zenodo to create an archive of your repository. Return to the Zenodo GitHub settings page to obtain your DOI badge, which you'll need for documentation.

5.1.2 Configuring Your Zenodo Entry

Open your entry on Zenodo and click the "Edit" button. You'll need to make several important modifications to ensure your package is properly categorized and accessible:

1. Locate the "Additional Notes" section and replace any existing text with a link to your GitHub pages website.
2. Adjust the upload type from "Software" to "Dataset" to properly categorize your submission. Under "Related identifiers", remove the current release link entry and add your GitHub pages URL, specifying that it "is compiled/created by this upload" and categorizing it as a Dataset.

5.2 Documentation Updates in RStudio

After configuring Zenodo, switch to RStudio. Update the version number in your DESCRIPTION file so it matches the release. Then write the DOI into the citation files (`CITATION.cff` and `inst/CITATION`) and the README with one call, using the DOI shown on your Zenodo entry:

```
update_citation(doi = "10.5281/zenodo.11185699")
```

The function regenerates both citation files from DESCRIPTION, adds the DOI badge to `README.Rmd`, rebuilds `README.md`, and rebuilds the website in `docs/` when it exists. Commit and push the changed files.

5.3 ETH Research Collection Submission

> **Note**
>
> This section applies only to GHE workflow submissions

For submissions to the ETH Research Collection, navigate to the research data section and select "dataset" as your submission type. Under organizational unit, select "tilley", and set your license to "Creative Commons Attribution 4.0 International". These settings ensure your submission aligns with institutional requirements and makes your data openly accessible while protecting your rights as the creator.

6 Troubleshooting

Although we try to make this guide as detailed as possible, unfortunately we cannot account for all eventualities. That's why we've compiled a list of common issues (and solutions) here.

6.1 Git fails when pushing commit to github

If you want to push large files (e.g. raw datasets) to GitHub, Git may fail. A simple workaround is to increase the buffer size. You can do this with the following line in the terminal. Note that you need to be in the working directory of your project.

```
git config http.postBuffer 524288000
```

6.2 R CMD CHECK

During the publication process, you might encounter several common issues when running R CMD CHECK. Here's how to address them:

If you receive a "unable to verify current time" error, this typically relates to your system's time synchronization. You can find a detailed solution in the Stack Overflow discussion addressing this specific issue.

Another common issue is the "Non-standard files/directories found at top level" warning. This occurs when your package contains files that R doesn't expect in the root directory. While this warning doesn't prevent your package from working, you can resolve it by following the guidance provided in this Stack Overflow thread.

Remember to verify all URLs and steps before beginning the process, as platforms occasionally update their interfaces and requirements. If you encounter any issues not covered in this manual, consult the respective platform's documentation or reach out to their support teams. By following these steps carefully, your data package is properly documented, citeable, and accessible to the research community.